

Data Partitioning & Optimizing Delta Lake



Delta Table Partitioning

Delta Table Partitioning

- You can partition data to speed up queries or DML that have predicates involving the partition columns.
- To partition data when you create a Delta table, specify a partition by columns.
- The most commonly used partition column is date.
- If the cardinality of a column will be very high, do not use that column for partitioning.
 - For example, if you partition by a column `userId` and if there can be 1M distinct user IDs, then that is a bad partitioning strategy.
- You can partition by a column if you expect data in that partition to be at least 1 GB.

<https://docs.delta.io/delta-batch/#partition-data>
<https://docs.delta.io/best-practices/>

Delta Table Partitioning (cont...)

```
df.write.format("delta").partitionBy("gender").saveAsTable("default.people10m")
```

```
DeltaTable.create(spark) \  
  .tableName("default.people10m") \  
  .addColumn("id", "INT") \  
  .addColumn("firstName", "STRING") \  
  .addColumn("middleName", "STRING") \  
  .addColumn("lastName", "STRING", comment = "surname") \  
  .addColumn("gender", "STRING") \  
  .addColumn("birthDate", "TIMESTAMP") \  
  .addColumn("ssn", "STRING") \  
  .addColumn("salary", "INT") \  
  .partitionedBy("gender") \  
  .execute()
```

Delta Table Partitioning

- To determine whether a table contains a specific partition, use the statement **SELECT COUNT(*) > 0 FROM <table-name> WHERE <partition-column> = <value>.**

- If the partition exists, true is returned. For example:

```
spark.sql(  
    "SELECT COUNT(*) > 0 AS `Partition exists`  
    FROM default.people10m WHERE gender = 'M'"  
)
```

- Delete Partition

```
DeltaTable.Delete  
    (F.col("<partition_column>")== "<value>")
```

Delta Table Partitioning

```
df = spark.createDataFrame(  
    [  
        ("Ernesto", "Guevara", "Argentina"),  
        ("Maria", "Sharapova", "Russia"),  
        ("Bruce", "Lee", "China"),  
        ("Jack", "Ma", "China"),  
    ]  
)
```

```
df.repartition(F.col("country"))
```

df.repartition Triggers a full shuffle so that rows with the same country value are clustered into the same Spark partition (task).

Delta Table Partitioning (cont...)

```
df.repartition(F.col("country")).write.partitionBy("country").format(
    "delta"
).saveAsTable("country_people")
```

```
spark-warehouse/country_people
├─ _delta_log
│   └─ 00000000000000000000000000000000.json
├─ country=Argentina
│   └─ part-00000-03ceafc8-b9b5-4309-8457-6e50814aaa8b.c000.snappy.parquet
├─ country=China
│   └─ part-00000-9a8d67fa-c23d-41a4-b570-a45405f9ad78.c000.snappy.parquet
└─ country=Russia
    └─ part-00000-c49ca623-ea69-4088-8d85-c7c2de30cc28.c000.snappy.parquet
```

Querying Delta Tables

Table Batch Reads and Writes

Spark SQL	Dataframe API in Pyspark
Create a table <code>CREATE TABLE bikeSharing USING DELTA LOCATION '/tmp/bikeSharing/'</code>	Create a table <code>df = spark.read.format("delta").load("/tmp/delta-table") df.write.format("delta").save("/tmp/delta-table")</code>
Partition Data <code>CREATE TABLE bikeSharingpartition USING DELTA PARTITIONED BY (mnth) LOCATION '/tmp/bikeSharing/'</code>	Partition Data <code>df = spark.read.format("delta").load("/tmp/delta-table") df.write.format("delta").partitionBy("date").save("/tmp/delta-table")</code>
Read Data <code>SELECT * FROM bikeSharing</code>	Read Data <code>df = spark.read.format("delta").load("/tmp/delta-table") df.show()</code>

<https://docs.delta.io/latest/delta-batch.html>

Table Batch Reads and Writes

Spark SQL	Dataframe API in Pyspark
Append <code>INSERT INTO bikesharing SELECT * FROM bikesharing_temp</code> Overwrite <code>INSERT OVERWRITE TABLE bikesharing SELECT * FROM bikesharing_temp</code> You can selectively overwrite only the data that matches predicates over partition columns <code>UPDATE bikesharing SET cnt = cnt+1 where dteday between "2011-01-01" and "2011-01-01"</code>	Append <code>df.write.format("delta").mode("append").save("/tmp/delta-table")</code> Overwrite <code>df.write.format("delta").mode("overwrite").save("/tmp/delta-table")</code> You can selectively overwrite only the data that matches predicates over partition columns <code>df.write .format("delta") .mode("overwrite") .option("replaceWhere", "date >= '2020-01-01' AND date <= '2020-01-31' ") .save("/tmp/delta-table")</code>

<https://docs.delta.io/latest/delta-batch.html>

Table Batch Reads and Writes

Update a Table – Fix incorrect updates

Spark SQL	Dataframe API in Pyspark
Fix the cnt column and increment it by one where date falls between "2011-01-01" and "2011-01-01" <pre>UPDATE bikesharing SET cnt = cnt+1 where dteday between "2011-01-01" and "2011-01-01"</pre>	You can update data that matches a predicate in a Delta table. For example, to fix a spelling mistake in the eventType, you can run the following: <pre>from delta.tables import * from pyspark.sql.functions import * deltaTable = DeltaTable.forPath(spark, "/data/events/") deltaTable.update("eventType = 'clck'", { "eventType": "'click'" }) # predicate using SQL formatted string /* deltaTable.update(col("eventType") == "clck", { "eventType": lit("click") }) # predicate using Spark SQL functions */</pre>

<https://docs.delta.io/latest/delta-batch.html>

Table Batch Reads and Writes

Update a Table - Upsert

Spark SQL	Dataframe API in Pyspark
<pre>MERGE INTO my_table target USING my_table TIMESTAMP AS OF date_sub(current_date(), 1) source ON source.userId = target.userId WHEN MATCHED THEN UPDATE SET *</pre>	<pre>deltaTable.alias("events").merge(updatesDF.alias("updates"), "events.eventId = updates.eventId") \ .whenMatchedUpdate(set = { "data": "updates.data" }) \ .whenNotMatchedInsert(values = { "date": "updates.date", "eventId": "updates.eventId", "data": "updates.data" }) \ .execute()</pre>

<https://docs.delta.io/latest/delta-update.html#-delta-update>

<https://docs.databricks.com/delta/delta-batch.html#language-sql>

Table Batch Reads and Writes

- Delete

You can remove data that matches a predicate from a Delta table

Spark SQL	Dataframe API in Pyspark
<pre>DELETE FROM events WHERE date < '2017-01-01' DELETE FROM delta.`/data/events/` WHERE date < '2017-01-01'</pre>	<pre>from delta.tables import * from pyspark.sql.functions import * deltaTable = DeltaTable.forPath(spark, "/data/events/") deltaTable.delete("date < '2020-01-01'") # <i>predicate using SQL formatted string</i> /* deltaTable.delete(col("date") < "2020-01-01") # <i>predicate using Spark SQL functions</i> */</pre>

<https://docs.delta.io/latest/delta-update.html#-delta-update>

Table Batch Reads and Writes

Read older versions of data using time travel

As you write into a Delta table or directory, every operation is automatically versioned. You can access the different versions of the data two different ways via version number or TimeStamp

Using Version

```
SELECT count(1) from bikeSharingDay VERSION AS OF 1  
SELECT count(1) from bikeSharingDay VERSION AS OF 0
```

Using timestamp

```
SELECT * FROM bikeSharingDay TIMESTAMP AS OF '2019-01-29 00:37:58'  
SELECT * FROM bikeSharingpartition TIMESTAMP AS OF '2019-01-29 00:37:58'
```

Introduction to time travel

<https://databricks.com/blog/2019/02/04/introducing-delta-time-travel-for-large-scale-data-lakes.html>

- timestamp_expression can be any one of:
 - '2018-10-18T22:15:12.013Z', that is, a string that can be cast to a timestamp
 - cast('2018-10-18 13:36:32 CEST' as timestamp)
 - '2018-10-18', that is, a date string
 - current_timestamp() - interval 12 hours
 - date_sub(current_date(), 1)
 - Any other expression that is or can be cast to a timestamp
- version is a long value that can be obtained from the output of DESCRIBE HISTORY table_spec.

Neither timestamp_expression nor version can be subqueries.

Exercise

Output all versions of your data

Pick one row and display all of that row's data in different versions

Optimizing Delta Tables

Vacuum

- **VACUUM deletes old data files that are no longer needed**—typically files left behind by:
 - updates (UPDATE)
 - deletes (DELETE)
 - merges (MERGE INTO)
 - Overwrites
 - Etc.
- These files remain in storage to support **Time Travel** and **streaming reads**.
- Because Delta Lake is **append-only** underneath.
- **VACUUM cleans these files up!**

<https://docs.delta.io/delta-utility/>

You can remove files no longer referenced by a Delta table and are older than the retention threshold by running the vacuum command on the table. vacuum is not triggered automatically. The default retention threshold for the files is 7 days.

vacuum deletes only data files, not log files. Log files are deleted automatically and asynchronously after checkpoint operations. The default retention period of log files is 30 days, configurable through the `delta.logRetentionDuration` property which you set with the `ALTER TABLE SET TBLPROPERTIES` SQL method.

The ability to [time travel](#) back to a version older than the retention period is lost after running vacuum.

Why Vacuum is needed!

- When you run a delete command, Delta **does NOT** remove the old Parquet files.
 - It writes *new* files without the row and marks the old ones as “removed”.
- Delta Lake is built on **Parquet**, which is a *columnar, immutable file format*. You *cannot* update a single row inside a Parquet file — the entire file must be rewritten. (**CoR → Copy on Write**)
 - Reads the files containing the rows that changed
 - Generates **new files** with the modifications Marks the old files as "removed" in the transaction log
 - Leaves them for Time Travel
 - VACUUM later physically removes them

MoR (Merge on Read)

- Instead of rewriting the data file immediately, the system writes a tiny "Delete Vector" or a "Change Log" file that says: *"Ignore row X in the original file; here is the new version of that row."*
- **This is done at Write Time:** Very fast. You only write the changed rows to a small new file.
- **While at Read Time:** The engine "merges" the original file and the change log in memory to show you the most recent state.
- **Compaction (The Clean-up):** Periodically, a OPTIMIZE process in Delta Lake takes all those small change files and the original large file and merges them into a clean, new Parquet file.

Table Utility commands

Vacuum

- Remove files no longer referenced by a Delta table and are older than the retention threshold
- The default retention threshold for the files is 7 days.
- Vacuum is not triggered automatically.

```
VACUUM [db_name.]table_name|path [RETAIN num HOURS] [DRY RUN]
```

```
%sql
```

```
VACUUM delta.`/data/events/` -- vacuum files not required by versions older than the default retention period
```

```
VACUUM delta.`/data/events/` RETAIN 100 HOURS -- vacuum files not required by versions more than 100 hours old
```

```
VACUUM delta.`/data/events/` DRY RUN -- Return a list of files to be deleted
```

<https://docs.databricks.com/spark/latest/spark-sql/language-manual/vacuum.html>

By default, retain hours for vacuum should be greater than 168 hour, which can be altered to a lower number with following setting :

spark.databricks.delta.retentionDurationCheck.enabled to false

Data retention

- To time travel, you must retain both the log and the data files for that version
- The data files are never deleted automatically. You need to run VACUUM for this to happen.
- Vacuum does not delete log files. Log files are automatically cleaned up after checkpoints are written
- By default you can time travel up to 30 days unless you have
 - Run Vacuum
 - Changed the data or log retention periods by using the following table.properties
 - **delta.logRetentionDuration** = "interval <interval>" (default 30 days)
 - **delta.deletedFileRetentionDuration** = "interval <interval>" (default 7 days)

Change Data Feed

- Tracks row level changes between versions of a Delta table
- 'Change events' are recorded by runtime for the data written to the table
 - These events consist of data plus metadata whether data was inserted, updated or deleted
- Silver To Gold, Transmit Changes, Audit Trail
- Enable by
 - `Create TABLE... TBLPROPERTIES (delta.enableChangeDataFeed = true)`
 - `ALTER TABLE <table> SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`
 - `spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;`

<https://docs.delta.io/delta-change-data-feed/>

Change Data Feed (CDF) feature allows Delta tables to track row-level changes between versions of a Delta table. When enabled on a Delta table, the runtime records “change events” for all the data written into the table. This includes the row data along with metadata indicating whether the specified row was inserted, deleted, or updated.

You can read the change events in batch queries using DataFrame APIs (that is, `df.read`) and in streaming queries using DataFrame APIs (that is, `df.readStream`).

Use cases

Change Data Feed is not enabled by default. The following use cases should drive when you enable the change data feed.

- **Silver and Gold tables:** Improve Delta performance by processing only row-level changes following initial MERGE, UPDATE, or DELETE operations to accelerate and simplify ETL and ELT operations.

- **Transmit changes:** Send a change data feed to downstream systems such as Kafka or RDBMS that can use it to incrementally process in later stages of data pipelines.
- **Audit trail table:** Capture the change data feed as a Delta table provides perpetual storage and efficient query capability to see all changes over time, including when deletes occur and what updates were made.

Change data storage

Delta Lake records change data for UPDATE, DELETE, and MERGE operations in the `_change_data` folder under the Delta table directory. These records may be skipped when Delta Lake detects it can efficiently compute the change data feed directly from the transaction log. In particular, insert-only operations and full partition deletes will not generate data in the `_change_data` directory.

The files in the `_change_data` folder follow the retention policy of the table. Therefore, if you run the [VACUUM](#) command, change data feed data is also deleted.

Read changes in batch queries

You can provide either version or timestamp for the start and end. The start and end versions and timestamps are inclusive in the queries. To read the changes from a particular start version to the *latest* version of the table, specify only the starting version or timestamp.

```
-- version as ints or longs e.g. changes from version 0 to 10
SELECT * FROM table_changes('tableName', 0, 10)
-- timestamp as string formatted timestamps
SELECT * FROM table_changes('tableName', '2021-04-21
05:45:46', '2021-05-21 12:00:00')
-- providing only the startingVersion/timestamp
SELECT * FROM table_changes('tableName', 0)
```

Optimize performances with file management

- To improve query speed, Delta Lake offers the ability to optimize the layout of data stored in storage
- Delta Lake supports two layout algorithms: **bin-packing** and **Z-Ordering**
- **Compaction (bin-packing)**
 - Bin-packing coalesces small files into larger ones, avoiding scanning many files when executing lookup queries
 - You trigger compaction by running the OPTIMIZE command

```
OPTIMIZE delta.`/data/events`
```
 - If you have a large amount of data and only want to optimize a subset of it, you can specify an optional partition predicate using **WHERE**

```
OPTIMIZE delta.`/data/events` WHERE date >= '2017-01-01'
```

<https://databricks.com/blog/2018/07/31/processing-petabytes-of-data-in-seconds-with-databricks-delta.html>

OPTIMIZE

<pre>%sql OPTIMIZE <u>scd2demo</u>;</pre>	
▶ (8) Spark Jobs	
▶ <code>_sqldf: pyspark.sql.dataframe.DataFrame = [path: string, metrics: struct]</code>	
Table ▾ +	
path	metrics
dbfs:/FileStore/tables/scd2demo	<pre>{ "numFilesAdded": 1, "numFilesRemoved": 5, "filesAdded": {"min": 2333, "max": 2333, "avg": 2333, "totalFiles": 1, "totalSize": 2333}, "filesRemoved": {"min": 2023, "max": 2073, "avg": 2033.4, "totalFiles": 5, "totalSize": 10167}, "partitionsOptimized": 0, "zOrderStats": null, "numBatches": 1, "totalConsideredFiles": 5, "totalFilesSkipped": 0, "preserveInsertionOrder": true, "numFilesSkippedToReduceWriteAmplification": 0, "numBytesSkippedToReduceWriteAmplification": 0, "startTimeMs": 1685894350328, "endTimeMs": 1685894353574, "totalClusterParallelism": 16, "totalScheduledTasks": 1, "autoCompactParallelismStats": null}</pre>

Optimize performances with file management- Continued

Data skipping

- As new data is inserted into a Delta table, file-level **min/max** statistics are collected for all columns of supported types
- When there's a lookup query against the table, Delta table first consults these statistics in order to determine which files can safely be skipped
- You do not need to configure data skipping - the feature is activated whenever applicable.
 - However, its effectiveness depends on the layout of your data. For best results, apply [Z-Ordering](#).

<https://databricks.com/blog/2018/07/31/processing-petabytes-of-data-in-seconds-with-databricks-delta.html>

Optimize performances with file management

Z-Ordering

- Z-Ordering is a [technique](#) to colocate related information in the same set of files
- This co-locality is automatically used by data-skipping algorithms to dramatically reduce the amount of data that needs to be read
- To Z-Order data, you specify the columns to order on in the ZORDER BY clause:

```
%sql  
OPTIMIZE events WHERE date >= current_timestamp() - INTERVAL 1 day  
ZORDER BY (eventType)
```

<https://databricks.com/blog/2018/07/31/processing-petabytes-of-data-in-seconds-with-databricks-delta.html>

Optimize performances with file management

Z-Ordering

- If you expect a column to be commonly used in query predicates and if that column has high cardinality, then use ZORDER BY
- You can specify multiple columns for ZORDER BY as a comma-separated list. However, the effectiveness of the locality drops with each additional column

<https://databricks.com/blog/2018/07/31/processing-petabytes-of-data-in-seconds-with-databricks-delta.html>

Employee Delta Table											
File 1				File 3				File 5			
Emp_id	Emp_name	Salary	Active	Emp_id	Emp_name	Salary	Active	Emp_id	Emp_name	Salary	Active
111	Michael	5000	Y	222	Nancy	5000	Y	555	Kevin	3000	N
555	Kevin	7000	Y	444	Tomas	6000	Y	888	Shane	4500	Y

Optimized Employee Table											
File 1				File 2				File 3			
Emp_id	Emp_name	Salary	Active	Emp_id	Emp_name	Salary	Active	Emp_id	Emp_name	Salary	Active
111	Michael	5000	Y	222	Nancy	5000	Y	555	Kevin	3000	N
555	Kevin	7000	Y	444	Tomas	6000	Y	888	Shane	4500	Y
333	David	4000	Y	333	David	2000	N	444	Tomas	2500	N
999	Peter	3000	Y	888	Shane	3000	N	999	Peter	2000	N

File	Min	Max
File1	111	999
File2	222	888
File3	444	999

select * from employee where emp_id =444	1,2,3	File	Min
select * from employee where emp_id <333	1,2	File1	111
		File2	222
		File3	444

Z-Ordered Employee Table											
File 1				File 2				File 3			
Emp_id	Emp_name	Salary	Active	Emp_id	Emp_name	Salary	Active				
111	Michael	5000	Y	444	Tomas	6000	Y				
222	Nancy	5000	Y	444	Tomas	2500	N				
333	David	4000	Y	555	Kevin	7000	Y				
333	David	2000	N	555	Kevin	3000	N				

select * from employee where emp_id =444	2	File	Min	Max
select * from employee where emp_id <333	1	File1	111	333
		File2	444	555
		File3	888	999


```

1 %sql
2 OPTIMIZE scd2Demo
3 ZORDER PK1

```

<https://medium.com/@amarkrgupta96/unlocking-performance-optimize-vacuum-and-z-ordering-in-databricks-delta-tables>

Table Utility commands - Contd

```
DESCRIBE HISTORY '/data/events/' -- get the full history of the table
DESCRIBE HISTORY '/data/events/' LIMIT 1 -- get the last operation only

DESCRIBE DETAIL <DATABASENAME.TABLENAME> -- Show table details

SHOW DATABASES -- List Databases
SHOW TABLES -- List tables
SHOW CREATE TABLE <DATABASENAME.TABLENAME> -- Show table creation script

RESTORE TABLE <DATABASENAME.TABLENAME> TO VERSION AS OF <version>

RESTORE TABLE '/data/target/' TO TIMESTAMP AS OF <timestamp>
```

<https://docs.delta.io/latest/delta-utility.html>

Knowledge Check

1. How can you optimize Delta lake performances
2. What is the impact of VACUUM table utility command

1. How can you optimize Delta lake performances?

- By using Bin-Packing and Zordering

2. What is the impact of VACUUM table utility command

Recursively vacuum directories associated with the Delta table and remove files that are no longer in the latest state of the transaction log for the table and are older than a retention threshold